



Microcontroladores

UNIVERSIDAD CATÓLICA DE
CUENCA
CARRERA DE ROBÓTICA E
INTELIGENCIA ARTIFICIAL

TEMA: CONTROL DE ACTUADORES
Y SECUENCIADORES LÓGICOS CON
ESP32

NOMBRE DEL ESTUDIANTE:
FRANCIS BENITEZ

DOCENTE: ING. PABLO BUESTÁN
ANDRADE, PhD

FECHA: 01-05-2026

INTRODUCCIÓN

La programación basada en interrupciones representa un paradigma avanzado en el desarrollo de software para microcontroladores. A diferencia del pooling tradicional, las interrupciones de hardware permiten que el ESP32 responda a eventos temporales con alta precisión. En esta práctica, se explora la implementación de una máquina de estados que gestiona la modulación por ancho de pulso (PWM) y secuencias digitales, garantizando un entorno de ejecución libre de funciones bloqueantes.

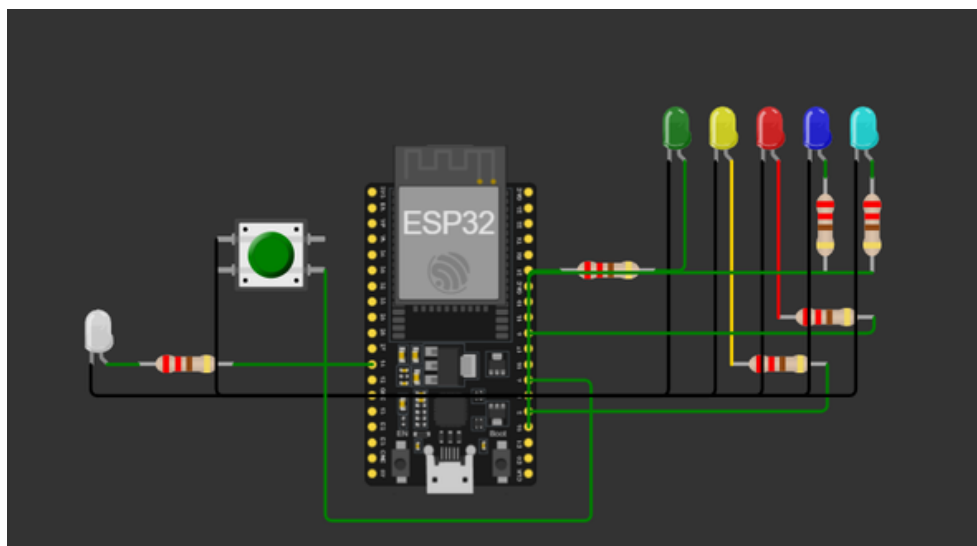
OBJETIVO GENERAL

Implementar un sistema de control de efectos lumínicos mediante el microcontrolador ESP32, utilizando temporizadores de hardware y modulación de ancho de pulso para la creación de comportamientos dinámicos sin el uso de retardos de software.

OBJETIVOS ESPECÍFICOS

- Configurar interrupciones de timer para la creación de una base de tiempo.
- Implementar el control de intensidad LED mediante `analogWrite()` sincronizado con la ISR.
- Desarrollar un secuenciador lógico de 5 niveles con tiempos de conmutación definidos por software no bloqueante.

DISEÑO ESQUEMÁTICO



```
#define LED_PWM 14
#define BOTON 4
#define LED1 15
#define LED2 2
#define LED3 5
#define LED4 18
#define LED5 19
```

Código

```
int leds[5] = {LED1, LED2, LED3, LED4, LED5};
volatile int counter = 0;
hw_timer_t *timer = NULL;
portMUX_TYPE timerMux = portMUX_INITIALIZER_UNLOCKED;
```

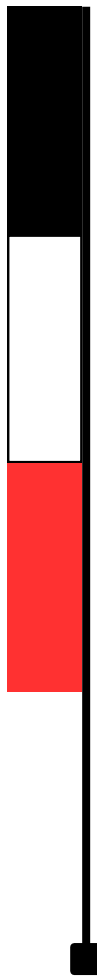
```
int estado = 1;
int ultimoEstadoBoton = HIGH;
int pwm = 0;
int direccionPWM = 1;
int ledActual = 0;
```

```
void IRAM_ATTR onTimer()
{
    portENTER_CRITICAL_ISR(&timerMux);
    counter++;
    portEXIT_CRITICAL_ISR(&timerMux);
}
```

```
void setup()
{
    Serial.begin(9600);
    pinMode(LED_PWM, OUTPUT);
    pinMode(BOTON, INPUT_PULLUP);
```

```
    for (int i = 0; i < 5; i++)
    {
        pinMode(leds[i], OUTPUT);
        digitalWrite(leds[i], LOW);
    }
```

```
    timer = timerBegin(1000000);
    timerAttachInterrupt(timer, &onTimer);
    timerAlarm(timer, 10000, true, 0);
}
```



```
void loop()
{
  int estadoBoton = digitalRead(BOTON);
  if (ultimoEstadoBoton == HIGH && estadoBoton == LOW)
  {
    if (estado == 1)
    {
      estado = 2;
      counter = 0;
      apagarTodos();
      analogWrite(LED_PWM, 0);
      Serial.println("Cambio a Estado 2: Ruleta Rusa");
    }
    else
    {
      estado = 1;
      counter = 0;
      apagarTodos();
      Serial.println("Cambio a Estado 1: Respiracion PWM");
    }
  }
  ultimoEstadoBoton = estadoBoton;
  if (estado == 1)
  {
    respiracionPWM();
  }
  else if (estado == 2)
  {
    ruletaRusa();
  }
}

void respiracionPWM()
{
  if (counter >= 2)
  {
    counter = 0;
    analogWrite(LED_PWM, pwm);
    Serial.print("PWM: ");
    Serial.println(pwm);
  }
}
```

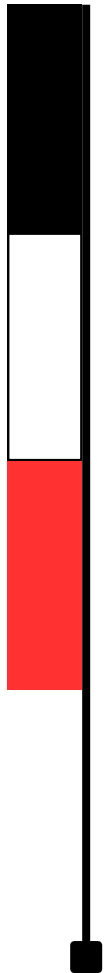
```
pwm = pwm + direccionPWM;
if (pwm >= 255)
{
    pwm = 255;
    direccionPWM = -1;
}
if (pwm <= 0)
{
    pwm = 0;
    direccionPWM = 1;
}
}
```

```
void ruletaRusa()
{
    if (counter >= 50)
    {
        counter = 0;
        int mascara = 1 << ledActual;
        for (int i = 0; i < 5; i++)
        {
            if (mascara & (1 << i))
            {
                digitalWrite(leds[i], HIGH);
            }
            else
            {
                digitalWrite(leds[i], LOW);
            }
        }
        Serial.print("LED encendido: ");
        Serial.println(ledActual + 1);

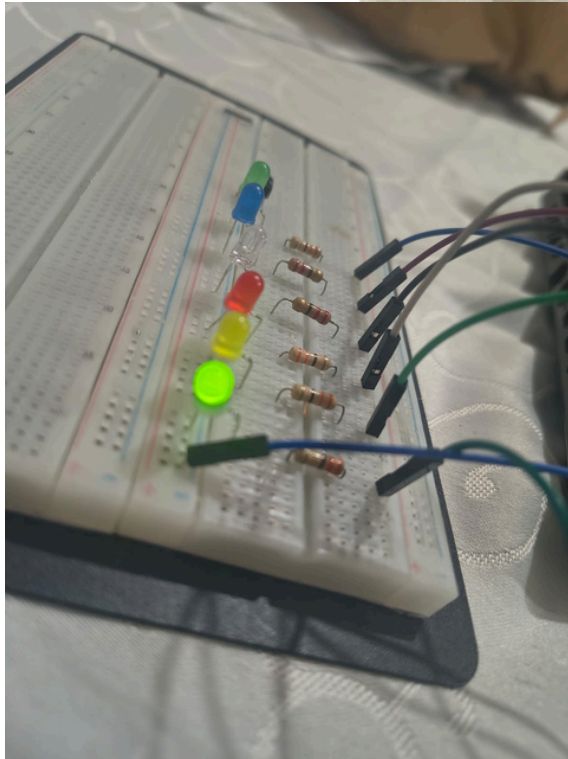
        ledActual++;

        if (ledActual >= 5)
        {
            ledActual = 0;
        }
    }
}

void apagarTodos()
{
    for (int i = 0; i < 5; i++)
    {
        digitalWrite(leds[i], LOW);
    }
}
```



https://drive.google.com/file/d/1NHrQsj1ujb_IKLUpERSN0aXNmTztK12i/view?usp=sharing



PWM: 218	PWM: 9
PWM: 219	PWM: 10
PWM: 220	PWM: 11
PWM: 221	PWM: 12
PWM: 222	PWM: 13
PWM: 223	PWM: 14
PWM: 224	PWM: 15
PWM: 225	PWM: 16
PWM: 226	PWM: 17
PWM	PWM: 18

En esta práctica el sistema funciona en dos modos que se cambian con un botón. En el primer estado, el LED principal hace un efecto de “respiración”, es decir, su brillo sube poco a poco hasta el máximo y luego baja nuevamente, repitiéndose de forma continua; además, en el monitor serial se puede ver cómo va cambiando el valor del PWM en cada paso. Cuando se presiona el botón, el sistema cambia al segundo estado, donde se encienden 5 LEDs uno por uno, como una secuencia tipo “coche fantástico”, y cada LED permanece encendido medio segundo antes de pasar al siguiente, sin que se prendan dos al mismo tiempo. Si se vuelve a presionar el botón, el sistema regresa al primer estado y repite el ciclo.

